

LEARNLYTICA •

MILESTONE 1

Candidate Preparation Guide

SQL • Python OOP • NumPy • Pandas

Concept-first preparation. Fresh practice scenarios.
Built to help you solve unfamiliar problems with confidence.

Candidate Preparation Material

Prepared for Learnlytica Milestone Readiness

Learn • Prove • Sustain

START HERE

How to use this guide

This handbook is deliberately scenario-neutral: it teaches the patterns you need without reproducing live assessment questions or answer keys.

KEY IDEA

What this guide does

It prepares you for the skills, decision patterns, boundary cases and coding habits that repeatedly matter in Milestone-style questions. Practice the pattern, not the sample.

The Milestone assessment expects you to read a requirement, preserve the requested function or query structure, implement the logic cleanly, and pass automated tests. The best preparation is therefore not memorizing one answer. It is learning to recognize the underlying pattern quickly.

Use the guide in three passes:

Pass	Goal	How to work
1. Learn	Understand the pattern	Read the explanation and worked example. Re-type the code rather than copying it.
2. Practice	Build recall	Solve the practice task without looking back. Compare only after you have a complete attempt.
3. Simulate	Build assessment confidence	Use a timer, work only in the required file/query window, run tests, fix failures, and stop when the requirements are satisfied.

ASSESSMENT HABIT

The rule that saves the most marks

Code from the written requirements, not from the sample output. Samples show one case; automated tests check general behavior.

The four technologies in this guide

Area	What you should be able to do confidently
SQL	Join related tables; aggregate correctly; use scalar subqueries; use LAG/RANK/DENSE_RANK; filter through subqueries/EXISTS; remove duplicates; sort deterministically.
Python OOP	Maintain dictionary-backed object state;

Area	What you should be able to do confidently
	add/update/get/filter records; use function parameters; raise exact exceptions; return the correct object level.
NumPy	Create typed arrays; validate size/type/range; compute statistics; categorize with vectorized logic; implement streak logic; use clipping/rounding safely.
Pandas	Create DataFrames with exact columns; clean invalid rows; preserve required dtypes; merge with the correct join; create flags; group/aggregate; apply thresholds; calculate summaries.

MINDSET

The assessment success loop

Use the same short workflow on every question. It reduces avoidable errors and keeps you moving.

1. Read the entire requirement once without coding.
2. Underline the exact input names, output names, required operation, thresholds, sort order, and error behavior.
3. Write the smallest correct solution. Avoid extra transformations that were not requested.
4. Run tests. Read the first failing message carefully and map it back to one requirement.
5. Fix only the cause. Re-run. Do not rewrite working parts unnecessarily.
6. Before submitting, verify signatures, dtypes, columns, ordering, thresholds, and return types.

WATCH OUT**Do not “improve” the specification**

If the task says keep a date as a string, keep it as a string. If it asks for a LEFT JOIN, use a LEFT JOIN. If it asks for a strict greater-than threshold, do not change it to greater-than-or-equal.

Fast requirement checklist

Check	Ask yourself
Signature	Did I keep the class/function/query structure exactly as given?
Input	Am I using the actual parameter rather than a sample value?
Output	Are the column names, tuple/list/dict shape and dtypes correct?
Logic	Did I use the requested operation (join type, window function, grouping, vectorization)?
Boundary	What happens exactly at the threshold?
Ordering	Is the result sorted exactly as requested?
Missing data	Should rows be kept, removed, or represented with NULL/NaN?
Errors	If an exception is required, is the type and message exact?

PART 1

SQL Pattern Mastery

The SQL questions are less about memorizing syntax and more about choosing the correct relationship, aggregation level and window/subquery pattern.

SQL pattern map

Pattern	Recognize it when the requirement says...	Primary tools
Previous/next record	previous date, prior transaction, next event	LAG / LEAD + PARTITION BY + ORDER BY
Highest/lowest row	the single order/invoice/score with maximum value	scalar subquery + MAX/MIN
Multi-table report	show names/details that live in several tables	JOIN on foreign keys
Average/total by entity	per course, per category, per customer	JOIN + GROUP BY + aggregate
Rank by count/value	rank entities, ties should share rank	RANK / DENSE_RANK
People who also did X	items used by users who performed a target action	subquery / EXISTS + DISTINCT

SQL 1

Previous-row analysis with LAG()

Use this when each row needs information from an earlier row belonging to the same entity.

Practice scenario: A courier company records package scans. For every scan, show the previous scan time for the same package. The first scan for a package should show NULL.

```
SELECT
  p.package_code,
  s.scan_id,
  s.scan_time,
  LAG(s.scan_time) OVER (
    PARTITION BY s.package_id
    ORDER BY s.scan_time, s.scan_id
  ) AS previous_scan_time
FROM scans s
JOIN packages p
  ON s.package_id = p.package_id
ORDER BY p.package_code, s.scan_time, s.scan_id;
```

How to reason about it

- **PARTITION BY** defines independent histories. Each package gets its own sequence.
- **ORDER BY inside OVER** defines which row is “previous.” Add a stable tie-breaker when two records can share a date/time.
- **LAG(column)** returns NULL automatically when there is no earlier row in that partition.
- **Final ORDER BY** controls display order; it is separate from the window ordering logic.

WATCH OUT**Common failure**

Using LAG() without PARTITION BY can pull a previous value from a different customer/package/entity. Another frequent failure is ordering only by a non-unique date, which can make tie behavior unstable.

PRACTICE**Mini drill**

Modify the query so it also shows the previous scan location using a second LAG(). Then test a package with two scans at the same timestamp.

SQL 2

Finding the row with the maximum value

When a problem specifically asks for a single-row subquery, solve it with a scalar subquery rather than replacing the technique with ORDER BY + LIMIT.

Practice scenario: A billing system needs the invoice with the largest invoice_total together with the client name.

```
SELECT
  i.invoice_id,
  c.client_name,
  i.invoice_total
FROM invoices i
JOIN clients c
  ON i.client_id = c.client_id
WHERE i.invoice_total = (
  SELECT MAX(invoice_total)
  FROM invoices
);
```

KEY IDEA**Why this works**

The inner query returns one scalar value: the maximum invoice total. The outer query returns the row or rows whose value equals that maximum.

BEST PRACTICE**Tie behavior**

If two rows share the same maximum, the equality form returns both. If the requirement truly guarantees a single maximum, that is fine. Do not silently add LIMIT 1 unless the specification asks for it.

Wrong direction when a subquery is explicitly required

```
-- Logically possible, but may violate the required technique
SELECT ...
FROM invoices
ORDER BY invoice_total DESC
LIMIT 1;
```

SQL 3

Multi-table JOIN reasoning

Build the relationship chain before writing the SELECT list. The most common join error is choosing the wrong path between entities.

Practice schema: Employees(employee_id, employee_name), Projects(project_id, project_name, department_id), Assignments(employee_id, project_id, role), Departments(department_id, department_name).

Task: Show each employee-project assignment, the employee name, project name, role, and the department responsible for the project.

```
SELECT
  e.employee_id,
  e.employee_name,
  p.project_id,
  p.project_name,
  a.role,
  d.department_name
FROM assignments a
JOIN employees e
  ON a.employee_id = e.employee_id
JOIN projects p
  ON a.project_id = p.project_id
JOIN departments d
  ON p.department_id = d.department_id;
```

ASSESSMENT
HABIT

Relationship test

Ask “which table owns the foreign key I need?” Here the department belongs to the project, so the department join must start from Projects, not from Employees.

JOIN type quick guide

Join	Use when	What happens to unmatched left rows
INNER JOIN	Only matched records should appear	Removed
LEFT JOIN	Every row from the left table must remain	Kept with NULLs on the right
RIGHT JOIN	Every row from the right table must remain	Depends on right-side preservation
FULL JOIN	Unmatched rows from both sides must remain	Kept from both sides

SQL 4

Aggregation across related tables

Many problems require you to join to the table containing the measure, then group at the level requested in the output.

Practice scenario: A training platform has Programs, Exercises and Attempts. Calculate the average attempt score for each program.

```
SELECT
  p.program_name,
  AVG(a.score) AS avg_score
FROM programs p
JOIN exercises e
  ON p.program_id = e.program_id
JOIN attempts a
  ON e.exercise_id = a.exercise_id
GROUP BY p.program_id, p.program_name
ORDER BY avg_score DESC;
```

KEY IDEA**One-row question**

Before GROUP BY, ask: “What does one output row represent?” If one row represents one program, group by the program identifier/name - not by learner or exercise.

WATCH OUT**Avoid accidental duplication**

Joining through an unnecessary table can multiply rows and change averages/sums. Only join tables required to reach the requested dimension and measure.

SQL 5

RANK() and DENSE_RANK()

Use a two-step pattern: aggregate first, then apply the ranking window function to the aggregated result.

Value	RANK()	DENSE_RANK()
100	1	1
100	1	1
90	3	2
80	4	3

Practice scenario: Rank technicians by number of completed service tickets in descending order.

```
WITH ticket_counts AS (
  SELECT
    technician_id,
    COUNT(*) AS ticket_count
  FROM service_tickets
  GROUP BY technician_id
)
SELECT
  t.technician_id,
  t.technician_name,
  c.ticket_count,
  RANK() OVER (ORDER BY c.ticket_count DESC) AS ticket_rank,
  DENSE_RANK() OVER (ORDER BY c.ticket_count DESC) AS dense_ticket_rank
FROM ticket_counts c
JOIN technicians t
  ON c.technician_id = t.technician_id
ORDER BY ticket_rank, t.technician_id;
```

BEST PRACTICE**Remember the difference**

RANK() leaves gaps after ties; DENSE_RANK() does not. If two people are rank 1, the next RANK is 3 but the next DENSE_RANK is 2.

SQL 6

“Users who also did X” queries

These problems test whether you can identify a qualifying set first, then use that set to retrieve related activity.

Practice scenario: List all products purchased by customers who have purchased the product named “Starter Kit”. Do not show duplicate products.

```
SELECT DISTINCT
  p.product_id,
  p.product_name
FROM purchases pu
JOIN products p
  ON pu.product_id = p.product_id
WHERE pu.customer_id IN (
  SELECT pu2.customer_id
  FROM purchases pu2
  JOIN products p2
    ON pu2.product_id = p2.product_id
  WHERE p2.product_name = 'Starter Kit'
);
```

**ASSESSMENT
HABIT****Three-step thinking**

1) Find qualifying customers. 2) Return all activity for those customers. 3) Use DISTINCT when the same result item could appear more than once.

Alternative using EXISTS

```
SELECT DISTINCT p.product_id, p.product_name
FROM purchases pu
JOIN products p ON pu.product_id = p.product_id
WHERE EXISTS (
  SELECT 1
  FROM purchases x
  JOIN products px ON x.product_id = px.product_id
  WHERE x.customer_id = pu.customer_id
    AND px.product_name = 'Starter Kit'
);
```

SQL LAB

SQL practice set

Solve these without copying the worked examples. Focus on relationship logic and exact output requirements.

7. Previous event: For each machine maintenance record, show the prior maintenance_date for the same machine. Sort by machine_code, maintenance_date, record_id.
8. Maximum row: Return the claim_id, member_name and claim_amount for the claim(s) equal to the maximum claim amount using a scalar subquery.
9. Multi-table report: Return consultant_name, project_name, allocation_percent and business_unit_name from Consultant, Allocation, Project and BusinessUnit tables.
10. Aggregation: Calculate average inspection_score per warehouse using Warehouse -> InspectionPlan -> InspectionResult.
11. Ranking: Rank agents by number of resolved cases, showing both RANK and DENSE_RANK.
12. Related behavior: Return distinct courses attended by learners who have attended a target course named "Python Foundations".

PRACTICE**Self-check before moving on**

For each SQL answer, identify the grain of the final row, the join keys, whether duplicates are possible, and whether ordering is part of correctness.

PART 2

Python OOP Pattern Mastery

The OOP tasks are usually small state-management systems. Precision matters more than advanced object-oriented design.

The reusable class pattern

```
class ExampleSystem:
    def __init__(self):
        self.records = {}

    def add_record(self, record_id, ...):
        ...

    def update_record(self, record_id, ...):
        ...

    def get_record(self, record_id):
        ...

    def filter_records(self, condition):
        ...
```

- **Dictionary key** is typically the unique ID.
- **Dictionary value** is usually another dictionary containing attributes.
- **Add** must reject duplicates when the requirement says IDs are unique.
- **Update** should modify only the requested field.
- **Get** often returns one nested dictionary, not the entire store.
- **Filter/list** must use the function parameter, not a hardcoded threshold or prefix.

OOP 1

Add, update and get records

Practice scenario: EquipmentCheckoutSystem manages devices by asset ID.

```
class EquipmentCheckoutSystem:
    def __init__(self):
        self.assets = {}

    def add_asset(self, asset_id: str, name: str, location: str) -> dict:
        if asset_id in self.assets:
            raise ValueError("Asset already exists")

        self.assets[asset_id] = {
            "name": name,
            "location": location,
            "status": "Available"
        }
        return self.assets

    def change_location(self, asset_id: str, new_location: str) -> dict:
        if asset_id not in self.assets:
            raise KeyError("Asset not found")

        self.assets[asset_id]["location"] = new_location
        return self.assets

    def get_asset_details(self, asset_id: str) -> dict:
        if asset_id not in self.assets:
            raise KeyError("Asset not found")

        return self.assets[asset_id]
```

WATCH OUT**Exact exceptions matter**

Automated tests can check both the exception type and the message. `ValueError("Asset already exists")` is not interchangeable with `Exception("Already exists")`.

BEST PRACTICE**Update only what was requested**

Do not rebuild the nested dictionary if you only need to change one field. Rebuilding can accidentally reset name/status values and fail state-preservation tests.

OOP 2

Filtering records with parameters

Practice scenario: Return IDs of workshop participants whose score is at least a caller-supplied minimum.

```
def qualified_participants(self, minimum_score: int) -> list:
    result = []

    for participant_id, details in self.participants.items():
        if details["score"] >= minimum_score:
            result.append(participant_id)

    return result
```

WATCH OUT**Hardcoding trap**

Never replace `minimum_score` with the sample value. Hidden tests typically call the same function with several thresholds specifically to check parameter use.

Prefix filtering pattern

```
def assets_by_zone(self, zone_prefix: str) -> list:
    result = []
    for asset_id, details in self.assets.items():
        if details["location"].startswith(zone_prefix):
            result.append(asset_id)
    return result
```

OOP LAB

Python OOP practice set

Use only the requested fields and exact return structure. Avoid extra validation unless a requirement explicitly asks for it.

13. Build `VehicleParkingSystem` with `self.vehicles = {}`. Add a vehicle with `status="Parked"`; duplicate ID should raise `ValueError("Vehicle already parked")`.
14. Add `move_slot(vehicle_id, new_slot)`. Missing ID should raise `KeyError("Vehicle not found")`. Only slot should change.
15. Add `get_vehicle_details(vehicle_id)` returning the nested vehicle dictionary.
16. Add `vehicles_by_floor(floor_prefix)` using an explicit loop and `startswith()`.
17. Build `CodingContestSystem` with `add_contestant()`, `update_score()`, `get_contestant_details()`, and `qualified_contestants(minimum_score)`.

**ASSESSMENT
HABIT****Minimalism is a strength**

Assessment code should be easy to inspect: direct conditions, direct dictionary updates, and direct returns. Extra abstractions can create more places for a mismatch.

PART 3

NumPy Pattern Mastery

NumPy tasks reward exact dtype, vectorized comparisons, correct range boundaries and careful return types.

NumPy pattern map

Skill	Typical tool
Create integer array	<code>np.array(values, dtype=np.int64)</code>
Check numeric dtype	<code>np.issubdtype(arr.dtype, np.number)</code>
Check invalid values	<code>np.any(...)</code> or <code>np.all(...)</code>
Statistics	<code>np.mean</code> / <code>np.std</code> / <code>np.max</code> / <code>np.min</code> / <code>np.sum</code>
Categorize values	<code>np.where</code> or <code>np.select</code>
Cap values	<code>np.clip</code>
Streaks	boolean condition + loop counters

NUMPY 1

Creation and validation

Practice scenario: A service desk records response times between 0 and 900 seconds.

```
import numpy as np

def create_time_array(values: list) -> np.ndarray:
    return np.array(values, dtype=np.int64)

def validate_time_array(arr: np.ndarray) -> bool:
    if arr.size == 0:
        return False

    if not np.issubdtype(arr.dtype, np.number):
        return False

    if np.any(arr < 0) or np.any(arr > 900):
        return False

    return True
```

BEST PRACTICE**Validation order**

Checking `arr.size` first avoids computing statistics or range checks on empty arrays. Then check `dtype`, then range.

**ASSESSMENT
HABIT****Inclusive ranges**

If valid values are 0 through 900 inclusive, 0 and 900 must pass. Only values below 0 or above 900 should fail.

NUMPY 2

Statistics and Python-native return types

Compute only the statistics requested and round at the exact stage specified.

```
def compute_time_stats(arr: np.ndarray) -> tuple:
    average = round(float(np.mean(arr)), 2)
    std = round(float(np.std(arr)), 2)
    maximum = int(np.max(arr))
    minimum = int(np.min(arr))
    return average, std, maximum, minimum
```

BEST PRACTICE**Why convert?**

NumPy operations can return NumPy scalar types such as `np.int64`. Some strict tests expect ordinary Python `int/float` objects, so explicit conversion is a safe habit when the specification describes Python scalar outputs.

Population vs sample standard deviation

`np.std(arr)` uses population standard deviation by default (`ddof=0`). Do not change to `ddof=1` unless the requirement explicitly asks for sample standard deviation.

NUMPY 3

Vectorized categorization and boundaries

Boundary values are a favorite automated-test target. Write the ranges on paper before writing np.where().

Range	Label
0-120	Fast
121-300	Normal
301-900	Slow
Outside range	Invalid

```
def categorize_times(arr: np.ndarray) -> np.ndarray:
    return np.where(
        (arr >= 0) & (arr <= 120),
        "Fast",
        np.where(
            (arr >= 121) & (arr <= 300),
            "Normal",
            np.where(
                (arr >= 301) & (arr <= 900),
                "Slow",
                "Invalid"
            )
        )
    )
```

PRACTICE

Boundary test set
Always test the exact edges: 0, 120, 121, 300, 301, 900, plus one invalid value below and above the valid range.

NUMPY 4

Longest consecutive streak

A streak is consecutive. Counting all matching values is not enough.

Practice scenario: Find the longest consecutive run of response times ≥ 301 seconds.

```
def longest_slow_streak(arr: np.ndarray) -> int:
    is_slow = arr >= 301
    current_streak = 0
    max_streak = 0

    for flag in is_slow:
        if flag:
            current_streak += 1
            max_streak = max(max_streak, current_streak)
        else:
            current_streak = 0

    return max_streak
```

WATCH OUT**Do not compare with the previous value**

A threshold streak asks whether each value satisfies a condition. It is different from an increasing streak, where you compare each value to the one before it.

Test mentally

```
Input condition: value >= 301
[100, 350, 420, 250, 390, 410, 405]
  ^  ^      ^  ^  ^
First streak = 2
Second streak = 3
Answer = 3
```

NUMPY 5

Bonus, clipping and grade/category mapping

Some array questions combine conditional updates with dtype conversion, percentage arithmetic, clipping and rounding.

```
def apply_bonus(scores: np.ndarray) -> np.ndarray:
    result = scores.astype(float).copy()
    mask = result > 80
    result[mask] = result[mask] * 1.05
    result = np.clip(result, None, 100)
    return np.round(result, 1)
```

BEST PRACTICE**Copy before mutation**

If the task should not modify the caller's original array, work on a copy. Also convert to float before applying a percentage when decimal results are expected.

Grade mapping using a loop

```
def to_grades(scores: np.ndarray) -> np.ndarray:
    grades = []
    for score in scores:
        if score >= 90:
            grades.append("A")
        elif score >= 80:
            grades.append("B")
        elif score >= 70:
            grades.append("C")
        else:
            grades.append("D")
    return np.array(grades)
```

NUMPY LAB

NumPy practice set

Complete these from scratch, then test boundary and empty-array cases.

18. Create np.int64 temperature array from a list.
19. Validate that an array is non-empty, numeric and every value is between -20 and 60 inclusive.
20. Return total, average rounded to 1 decimal, maximum and minimum.
21. Categorize temperatures into Cold (<10), Normal (10-29), Hot (30-60), Invalid (outside range).
22. Find the longest consecutive Hot streak.
23. Apply a 10% adjustment to values above 40, cap at 60 and round to one decimal.

PART 4

Pandas Pattern Mastery

Pandas questions combine exact schema handling with cleaning, left joins, derived columns, grouping and strict threshold filters.

The golden rule

KEY IDEA

Preserve the requested schema

Column names, capitalization, order and dtypes can be part of the tests. Avoid “helpful” renaming or date conversion unless the specification requests it.

PANDAS 1

Create a DataFrame with exact columns

Practice scenario: A delivery company tracks route sessions.

```
import pandas as pd

class DeliveryAnalyzer:
    def create_deliveries_df(self, data: list) -> pd.DataFrame:
        columns = [
            "DeliveryID",
            "HubID",
            "City",
            "DeliveryDate",
            "DistanceKm",
            "DurationMinutes",
            "PaymentStatus"
        ]
        return pd.DataFrame(data, columns=columns)
```

WATCH OUT**Date dtype trap**

If the requirement says DeliveryDate must remain a string, do not call `pd.to_datetime()`. Exact dtype expectations are common in automated tests.

PANDAS 2

Cleaning invalid records

Clean in explicit stages so each rule is visible and easy to debug.

```
def clean_deliveries_data(self, df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.dropna(
        subset=["DeliveryID", "HubID", "City", "PaymentStatus"]
    )

    cleaned = cleaned[cleaned["DistanceKm"] > 0]
    cleaned = cleaned[cleaned["DurationMinutes"] > 0]
    cleaned = cleaned[
        cleaned["PaymentStatus"].isin(["Paid", "Pending", "Failed"])
    ]

    return cleaned.reset_index(drop=True)
```

BEST PRACTICE**Use subset= in dropna()**

Dropping every row with any missing value may remove rows the specification intended to keep. Drop nulls only from required fields unless told otherwise.

PANDAS 3

Binary flags with strict thresholds

Convert the boolean condition to integer when the expected output is 0/1.

```
def add_long_delivery_flag(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
    df["IsLongDelivery"] = (
        df["DurationMinutes"] > threshold
    ).astype(int)
    return df
```

**ASSESSMENT
HABIT****Strict means strict**

If the wording says “greater than threshold,” a value equal to the threshold must produce 0. Test exactly equal, one below and one above.

PANDAS 4

GroupBy summaries

Named aggregation produces clear output column names and avoids extra rename steps.

```
def hub_utilization_summary(self, df: pd.DataFrame) -> pd.DataFrame:
    result = (
        df.groupby("HubID")
        .agg(
            SessionCount=("DeliveryID", "count"),
            TotalDistanceKm=("DistanceKm", "sum"),
            AverageDuration=("DurationMinutes", "mean")
        )
        .reset_index()

    result["AverageDuration"] = result["AverageDuration"].round(1)
    return result
```

BEST PRACTICE**Count the right thing**

When each row is one session, size() or count of a non-null ID can both work. If nulls are possible in the counted column, size() and count() behave differently - follow the requirement.

PANDAS 5

LEFT merge and returned/not-returned flags

Use a left merge when every record from the primary dataset must remain even if there is no match in the secondary dataset.

Practice scenario: Merge service jobs with warranty claims. Not every service job has a warranty claim.

```
def merge_jobs_claims(self, jobs_df, claims_df):  
    return pd.merge(  
        jobs_df,  
        claims_df,  
        on="JobID",  
        how="left"  
    )
```

```
merged["HasClaim"] = merged["ClaimAmount"].notna().astype(int)
```

WATCH OUT**Why INNER JOIN is wrong here**

An inner merge would remove jobs that do not have a claim. If the analysis needs both claimed and unclaimed jobs, that changes the denominator and breaks later rates.

PANDAS 6

Rates, thresholds and filtered summaries

Build the summary first, then apply threshold filters to the aggregated result.

```
def category_claim_rate(self, merged_df):
    merged_df["HasClaim"] = merged_df["ClaimAmount"].notna().astype(int)

    result = (
        merged_df.groupby("Category")
        .agg(
            Jobs=("JobID", "size"),
            ClaimedJobs=("HasClaim", "sum")
        )
        .reset_index()

    result["ClaimRate"] = result["ClaimedJobs"] / result["Jobs"] * 100
    return result.sort_values("Category").reset_index(drop=True)
```

```
def high_claim_suppliers(self, merged_df, n):
    returned = merged_df[merged_df["HasClaim"] == 1]
    counts = (
        returned.groupby("SupplierID")
        .size()
        .reset_index(name="ClaimCount")
    )
    return counts[counts["ClaimCount"] > n].reset_index(drop=True)
```

ASSESSMENT HABIT

Filter after aggregation

If *n* refers to “number of returns/claims/sessions,” compute the count first and then compare *Count* > *n*. Filtering raw rows with a threshold intended for group counts is a different operation.

PANDAS 7

Revenue and derived calculations

Filter qualifying rows first, create the derived value, then group.

```
def city_revenue_summary(self, df: pd.DataFrame) -> pd.DataFrame:
    paid = df[df["PaymentStatus"] == "Paid"].copy()
    paid["Revenue"] = paid["DistanceKm"] * 15

    result = (
        paid.groupby("City")["Revenue"]
            .sum()
            .reset_index()
    )
    return result
```

WATCH OUT**Order of operations**

If only Paid rows contribute to revenue, filter before calculating/grouping. Including Pending or Failed rows can produce plausible-looking but incorrect totals.

PANDAS LAB

Pandas practice set

Build one class with independent methods. Each method should work when tested on its own with a suitable DataFrame.

24. Create `MaintenanceAnalyzer.create_logs_df()` with columns `LogID`, `MachineID`, `Site`, `LogDate`, `RuntimeHours`, `DowntimeMinutes`, `Status`. Keep `LogDate` as string.
25. Clean null required IDs/site/status; keep `RuntimeHours > 0` and `DowntimeMinutes >= 0`; allow statuses `Completed`, `Pending`, `Failed`; reset index.
26. Add `IsMajorDowntime = 1` when `DowntimeMinutes > threshold`, else 0.
27. Summarize per `MachineID`: `LogCount`, `TotalRuntime`, `AverageDowntime` rounded to one decimal.
28. Return machines where `TotalRuntime > runtime_threshold`.
29. Calculate `SiteCost` from `Completed` rows only using `RuntimeHours * 20`.

PART 5

Why correct-looking solutions still fail

Automated tests are often checking details that sample outputs do not make obvious.

Failure pattern	What the test may be checking	Correction
Wrong threshold edge	> versus >=	Test equal/below/above explicitly
Wrong join	Row preservation	Translate requirement into INNER/LEFT behavior
Hardcoded sample value	Function parameter usage	Use the argument everywhere the rule depends on it
Wrong dtype	int64/string/float expectation	Create/convert only as required
Wrong return level	full dictionary vs one record	Read the return example/description carefully
Wrong ordering	deterministic sorted output	Use every requested sort key and direction
Wrong columns	exact schema names/order	Avoid unnecessary renames or extra columns
Index mismatch	expected 0..n-1 index	reset_index(drop=True)
Duplicate results	one-to-many joins/history	Use DISTINCT when requirement says unique
Wrong streak logic	total matches vs consecutive matches	Use reset-to-zero counter logic

DEBUGGING

How to use test failures intelligently

Treat a test failure as evidence about one requirement. Do not panic and rewrite the entire solution.

30. Read the failure message and identify the function/query being tested.
31. Compare actual vs expected shape first: type, rows, columns, tuple/list/dict.
32. Check boundaries and filters next.
33. Check ordering and index if values look correct but comparison still fails.
34. Check dtype if printed values look identical.
35. Check whether the test uses a different function argument than the sample.
36. Re-run after one focused change.

**ASSESSMENT
HABIT****Printed output can be misleading**

A Pandas/NumPy object can print the same values while having a different dtype, index or column order. Automated tests can see those differences.

PART 6

Mini Mock Milestone

Use this as a timed rehearsal. These are new scenarios designed around the same skill families, not copies of assessment questions.

Suggested timing: 75-90 minutes. Do not look at the answer guide until you finish all tasks.

Task 1 - SQL window function

Tables: Accounts(account_id, account_name), Payments(payment_id, account_id, payment_date, amount). Return account_name, payment_id, payment_date and previous_payment_date for the same account. First payment should show NULL. Sort by account_name, payment_date, payment_id.

Task 2 - SQL scalar subquery

Tables: Members(member_id, member_name), Claims(claim_id, member_id, claim_amount). Return claim_id, member_name and claim_amount for the maximum claim using a single-row subquery.

Task 3 - SQL rank

Tables: Agents(agent_id, agent_name), Cases(case_id, agent_id). Count cases per agent and return RANK and DENSE_RANK in descending case count order.

Task 4 - Python OOP

Create WorkshopSystem with self.members = {}. Implement add_member(member_id, name, score), update_score(member_id, new_score), get_member_details(member_id), and qualified_members(minimum_score). Duplicate member: ValueError("Member already exists"). Missing member: KeyError("Member not found"). Status should be "Active".

Task 5 - NumPy

Implement module-level functions to create np.int64 sensor arrays, validate values 0-1000 inclusive, compute mean/std/max/min, categorize 0-100 Normal, 101-300 Elevated, 301-1000 Critical, and find longest consecutive Critical streak.

Task 6 - Pandas cleaning

Create FleetAnalyzer.create_trips_df() with TripID, VehicleID, City, TripDate, DistanceKm, DurationMinutes, Status. Keep TripDate as string. Clean required fields, positive distance/duration and valid statuses Completed/Pending/Failed.

Task 7 - Pandas aggregation

For cleaned trip data, add IsLongTrip for DurationMinutes > threshold. Summarize VehicleID into TripCount, TotalDistanceKm and AverageDuration rounded to one decimal. Return vehicles whose TotalDistanceKm is strictly greater than a supplied threshold.

Task 8 - Pandas revenue

Using the same trip DataFrame, calculate revenue only for Completed rows as DistanceKm * 10, then return City and Revenue totals.

Mock execution checklist

Task	Before you move on
1-3 SQL	Check required technique, join path, aliases and final ordering.
4 OOP	Check exact exception type/message, state preservation and parameter use.
5 NumPy	Check dtype, inclusive boundaries, rounding and streak reset behavior.
6-8 Pandas	Check exact columns, string date, reset index, strict thresholds and qualifying status filter.

PRACTICE

Simulation rule
Do not open the answer section until you have a complete attempt for every task. Mark uncertain items and return to them after your first pass.

MOCK REVIEW

Self-marking answer patterns

Use these patterns to review your approach. Your exact aliases may vary only if the task permits them.

Task 1 pattern

```
LAG(payment_date) OVER (
  PARTITION BY account_id
  ORDER BY payment_date, payment_id
)
```

Task 2 pattern

```
WHERE claim_amount = (
  SELECT MAX(claim_amount)
  FROM Claims
)
```

Task 3 pattern

```
WITH counts AS (... GROUP BY agent_id)
SELECT ...,
  RANK() OVER (ORDER BY case_count DESC),
  DENSE_RANK() OVER (ORDER BY case_count DESC)
FROM counts ...
```

Review checklist for Tasks 4-8

Area	Must be true
Task 4 - OOP	self.members dictionary; minimum_score parameter used; exact duplicate/missing exceptions; update changes only score; get returns nested dictionary.
Task 5 - NumPy	np.int64 array; empty/numeric/range validation; inclusive boundaries; rounded statistics; consecutive streak reset logic.
Tasks 6-8 - Pandas	Exact columns and string date; dropna only on required fields; strict > thresholds; reset_index(drop=True); Completed rows only for revenue.

ASSESSMENT HABIT

Review the requirement, not your memory

If your solution differs from these patterns, that can still be correct only when it satisfies the exact stated requirement. The test is the specification.

PART 7

Cheat Sheet - SQL

Use this for revision, not as a substitute for understanding the pattern.

Need	Template
Previous value	LAG(col) OVER (PARTITION BY entity ORDER BY date, id)
Maximum row	WHERE value = (SELECT MAX(value) FROM table)
Aggregate	SELECT group_col, AVG/SUM/COUNT(measure) ... GROUP BY group_col
Rank	RANK() OVER (ORDER BY metric DESC)
Dense rank	DENSE_RANK() OVER (ORDER BY metric DESC)
Unique output	SELECT DISTINCT ...
Qualifying set	WHERE id IN (SELECT id FROM ... WHERE condition)
Preserve left rows	LEFT JOIN right ON ...

SQL final check

- Correct table relationships
- Correct group level
- Correct window partition/order
- Correct tie handling
- Correct DISTINCT usage
- Exact output aliases
- Exact final sort

CHEAT SHEET

Python OOP

The simplest correct class is usually the strongest assessment answer.

```
class System:
    def __init__(self):
        self.records = {}

    def add(self, rid, ...):
        if rid in self.records:
            raise ValueError(...)
        self.records[rid] = {...}
        return self.records

    def update(self, rid, value):
        if rid not in self.records:
            raise KeyError(...)
        self.records[rid]["field"] = value
        return self.records

    def get(self, rid):
        if rid not in self.records:
            raise KeyError(...)
        return self.records[rid]
```

- Do not change the constructor if one is prescribed
- Do not hardcode sample IDs or thresholds
- Raise exact required errors
- Update only the requested field
- Return the expected dictionary level/list

CHEAT SHEET

NumPy

Think dtype -> validation -> vectorized calculation -> exact return type.

```
arr = np.array(values, dtype=np.int64)
arr.size == 0
np.issubdtype(arr.dtype, np.number)
np.any(arr < low)
np.any(arr > high)
np.mean(arr)
np.std(arr)
np.max(arr)
np.min(arr)
np.clip(arr, low, high)
np.round(arr, decimals)
np.where(condition, value_if_true, value_if_false)
```

- Test exact range edges
- Convert NumPy scalar to int/float if required
- Use counters for consecutive streaks
- Use `.astype(float)` before percentage updates
- Do not use Pandas in NumPy-only tasks

CHEAT SHEET

Pandas

Keep transformations readable and independent.

```
pd.DataFrame(data, columns=[...])
df.dropna(subset=[...])
df[df["x"] > threshold]
df[df["status"].isin([...])]
df.reset_index(drop=True)
pd.merge(left, right, on="ID", how="left")
df["Flag"] = condition.astype(int)
df.groupby("Key").agg(...).reset_index()
df.sort_values([...], ascending=[...])
```

- Keep requested date/string dtype
- Use LEFT merge when unmatched primary rows must remain
- Filter valid/paid/completed rows before summaries when required
- Round only the column/precision requested
- Apply strict threshold to the aggregated metric when that is what n represents

FINAL PREPARATION

48-hour readiness plan

Use short focused practice instead of repeatedly rereading solutions.

When	Focus	What to do
48-24 hours before	Pattern recall	Solve 2 SQL, 1 OOP, 1 NumPy and 1 Pandas task without notes. Review only after completing each attempt.
24-12 hours before	Weak areas	Repeat the patterns where you made boundary, join, dtype or return-shape mistakes.
12-2 hours before	Light revision	Review cheat sheets, exact exception habits, threshold rules and DataFrame/NumPy syntax. Avoid learning new techniques.
Assessment start	Requirement extraction	Spend the first minute per question identifying signature, required operation, output and edge cases.
During tests	Focused debugging	Fix one failure cause at a time. Do not replace a mostly correct solution with a new design.
Before submit	Final scan	Check exact names, dtype, threshold, join type, sorting, reset_index and return type.

KEY IDEA

Confidence comes from pattern recognition
A new business story should not feel like a new programming problem. Reduce it to the pattern: previous row, maximum row, join chain, group-by, rank, dictionary state, validation, categorization, streak, cleaning, merge or aggregation.

Assessment-day checklist

Read this once immediately before starting.

✓	Check
☐	I will not change required class/function names or parameters.
☐	I will use function arguments, not sample values.
☐	I will preserve exact column names and required dtypes.
☐	I will check > versus >= at every threshold.
☐	I will use the requested join/window/subquery technique.
☐	I will reset Pandas indexes when required.
☐	I will remove duplicates only when the requirement calls for unique output.
☐	I will distinguish consecutive streaks from total counts.
☐	I will read test failures before changing code.
☐	I will stop when the specification is satisfied instead of adding unnecessary logic.

LEARNLYTICA

Learn • Prove • Sustain

Milestone 1 Candidate Preparation Guide